

Dropout-Inspired Strategies for Enhanced Exploration in Evolutionary Algorithms

Daniele Ganci

LASIGE, Faculty of Sciences, University of Lisbon
Lisbon, Portugal
University of Bologna
Bologna, Italy
fc65144@alunos.ciencias.ulisboa.pt
daniele.ganci2@unibo.it

Luís Correia

LASIGE, Faculty of Sciences, University of Lisbon
Lisbon, Portugal
lmcorreia@ciencias.ulisboa.pt

Abstract

Finding the right balance between exploration and exploitation is a crucial challenge in evolutionary algorithms (EAs), as it strongly impacts solution quality and the ability to escape local optima. While diversity-preserving mechanisms are commonly employed, high population diversity does not necessarily translate into improved performance across problem landscapes. Recent empirical evidence suggests that effective EA optimization largely depends on predominantly exploitative search dynamics, with exploration being most beneficial during the early iterations. Nevertheless, sporadic exploratory bursts may help reintroduce diversity and enhance solution quality. In this study, inspired by dropout regularization in neural networks (NNs), we introduce three mechanisms—and their variants—that stochastically deactivate entire individuals or subsets of decision variables with the expectation of enhancing exploration. We assess the effectiveness of dropout in genetic algorithms (GAs) on twelve single-objective and five multi-objective benchmark functions. We empirically demonstrate that per-variable stochastic deactivation, termed Individual Dropout (IDrop), significantly outperforms a standard GA on single-objective problems with exploitable global structure and improves convergence toward the theoretical Pareto front in multi-objective settings. We also show that, while dropout alone is not exempt from the curse of dimensionality, combinations of dropout mechanisms can partially alleviate this limitation in most of the tested functions.

CCS Concepts

• Computing methodologies → Genetic algorithms.

Keywords

Computer aided/automated design, Genetic algorithms, Heuristics, Noisy optimization, Empirical study

ACM Reference Format:

Daniele Ganci and Luís Correia. 2026. Dropout-Inspired Strategies for Enhanced Exploration in Evolutionary Algorithms. In *Genetic and Evolutionary Computation Conference (GECCO '26)*, July 13–17, 2026, San Jose, Costa Rica. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3795095.3805162>



This work is licensed under a Creative Commons Attribution 4.0 International License. *GECCO '26, San Jose, Costa Rica*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2487-9/2026/07

<https://doi.org/10.1145/3795095.3805162>

1 Introduction

Achieving and maintaining an effective balance between exploration and exploitation is a fundamental requirement for the performance of evolutionary algorithms (EAs). As highlighted by Črepinšek et al. [29], exploration can be informally seen as "the process of visiting entirely new regions of a search space", while exploitation as "the process of visiting those regions of a search space within the neighborhood of previously visited points". The ability to increase exploration when necessary—while appropriately limiting exploitation—and vice versa, has a direct impact on the algorithm's capacity to identify globally optimal solutions. For example, in the context of genetic algorithms (GAs), Michalewicz [14] describes this class of general-purpose, domain-independent search methods as achieving "a remarkable balance between exploration and exploitation within the search space." Consequently, the design of mechanisms explicitly aimed at regulating this balance has long been a focus of research.

Diversity-promoting strategies represent one of the most widely explored approaches for balancing such a ratio. Without appropriate diversity handling methods—namely, diversity maintenance, control or learning [22]—the population often converges rapidly toward high-fitness regions that may be suboptimal, a phenomenon commonly described as premature convergence [16, 23]; as the population becomes confined to local optima, it tends to lack the genetic variability required to escape and further explore the search space. Nevertheless, empirical studies indicate that maintaining high population diversity does not necessarily guarantee improved performance in EAs for all problems [7, 20]. For example, it is reasonable to expect that the optimization of instances characterized by relatively smooth or well-behaved fitness landscapes might actually suffer from such mechanisms.

Moreover, the exact relationship between diversity, exploration and exploitation is still unclear. As a rule of thumb, population tends to lose diversity as the generations go on [29]. High diversity enables exploration and serves as an indicator that the evolutionary process is operating in explorative regions, whereas low diversity is associated with exploitative ones.

Progresses have been made in determining at what moment the procedure *needs* to operate in one of the two spaces. Morales-Castañeda et al. [15] conducted an extensive empirical analysis comparing the behavior of 12 evolutionary and swarm-based algorithms across 43 well-known benchmark functions. Their results indicate that the best overall performance is typically achieved when the search process maintains an approximate balance of 10%

exploration and 90% exploitation [15]. This suggests the possibility of gradually introducing decreasing amounts of diversity following an exponential schedule, while maintaining low exploitation during the early stages of the optimization process.

An additional interesting observation arises from the results of the Whale Optimization Algorithm (WOA), which appears to achieve the best performance indicators in most of the non-shifted test functions due to its intrinsic characteristics, which lead to a particularly rough balance response. This response is characterized by non-uniform fluctuations arising from abrupt variations in population diversity. Moreover, these fluctuations tend to be larger during the initial iterations and gradually diminish over time. Such dynamics enable the algorithm to escape local minima, even when the search predominantly operates within exploitative regions of the solution space [15]. This suggests that inducing non-uniform exploration peaks during the calibration of the exploration–exploitation balance may enhance the quality of the obtained solutions. Nevertheless, WOA exhibits poor performance on shifted functions due to its intrinsic search dynamics, which hampers the identification and exploitation of promising regions of the search space.

Premature stagnation in local optima is often the consequence of an inadequate handling of regions that require intensified exploration, where the metaheuristic fails to maintain the appropriate balance between exploration and exploitation. As a result, the algorithm may waste a large number of function evaluations ineffectively, typically when corrective exploratory actions are no longer timely. Over the years, substantial efforts have been devoted to the development of mechanisms to mitigate such an obstacle [29].

Motivated by the findings of Morales-Castañeda et al. [15], our aim is to design and analyze a novel mechanism that enhances the exploratory capability of the search process throughout its execution. To this end, we introduce three diversity-induction mechanisms—and their respective variants—inspired by the dropout regularization operator [10] widely adopted in neural networks (NNs). These inject uncoupled exploratory bursts that perturb the solution space by probabilistically deactivating entire individuals or subsets of decision variables, thereby promoting sustained diversity and reducing the risk of premature convergence.

2 Dropout-based strategies

Dropout is a common regularization technique in NNs, which consists of deactivating a random subset of computational units at each training epoch by setting their output to 0, thereby encouraging the remaining units to learn more robust representations and reducing the risk of overfitting [10]. This technique is feasible and effective mainly because NNs naturally contain a certain amount of redundancy, particularly when they comprise a large number of units. Temporarily excluding part of the network at each epoch does not therefore impair learning; rather, it promotes the emergence of different sub-networks, which in turn encourage the remaining units to capture and generalize the relevant patterns in the data.

Unlike NNs, whose primary objective is representation learning, EAs pursue the identification of high-quality solutions to complex optimization problems under limited computational budgets. In addition, while EAs have some redundancy at the population level,

they usually do not have any at the individual level, meaning in the genetic representation.

It should be noted that this random perturbation is not related to the fitness of each individual or to the fitness distribution of the population. It is also not a type of mutation since it does not generate genetic changes that are inherited by descendants. Nevertheless, we expect that a stochastic deactivation of entire individuals or of decision variables within individuals, on a per-generation basis, might serve as an effective perturbation mechanism to enhance exploratory behavior in the search space when needed. It may effectively contribute to the identification and promotion of individuals who have relevant building blocks in the genome.

Thus, we introduce the following three main mechanisms—and their variants—which are described in detail in the next subsections:

- *Individual Dropout* (IDrop): random deactivation of subsets of decision variables in every individual prior to evaluation in each generation.
- *Population Dropout* (PDrop): random inhibition of individuals in each generation.
- Combinations of the previous strategies, in three alternatives (namely, I&PDrop1, I&PDrop2 and I&PDrop3)

The per-generation exclusion of a subset of components—either decision variables or entire individuals—is performed through a stochastically generated Boolean mask, constructed at each iteration according to a predefined dropout rate r . Thus,

$$p = 1 - r \quad (1)$$

is the probability of not dropping a component.

Each method is based on a specific rate, namely:

- r_i is the rate at which a variable is dropped within an individual.
- r_p is the rate at which an entire individual is dropped from the evolutionary cycle.
- r_{pe} is the rate at which an individual is exempted from per-variable dropout.

We hypothesize that such mechanisms may be particularly beneficial for optimization problems characterized by challenging fitness landscapes, rich in local optima where the search process is more prone to stagnation.

2.1 Individual Dropout

IDrop works by stochastically deactivating subsets of decision variables—i.e., randomly setting them to 0 or 1 depending on whether their contribution to the fitness function is additive or multiplicative, respectively, according to r_i —within each individual prior to evaluation at every generation, while always keeping at least two components active. Consequently, each individual gets a noisy fitness that is used during the subsequent selection phase. After variation operators are applied to the non-dropped variables, individuals are re-evaluated using the complete set of decision variables.

The key idea of this strategy is that the temporary exclusion of decision variables subtly modulates the selection pressure, helping the search to escape potential stagnation. It is reasonable to expect that such an approach would be particularly effective on problems with rugged fitness landscapes, such as those that are highly multimodal or of high dimensionality.

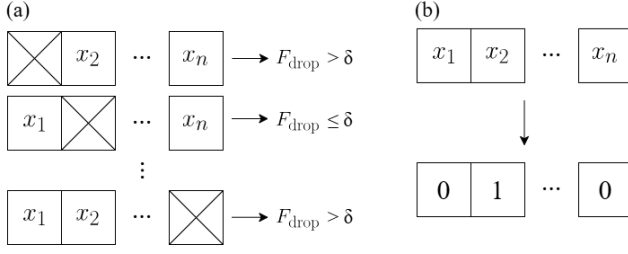


Figure 1: (a) Preparation phase: neutral values are assigned based on a fitness-drop threshold ($\delta = 10^{-6}$); (b) Neutral-value substitution.

Finding the right neutral value is challenging, as it should ideally contribute null effect to the fitness. To handle this, we employ a “preparation” phase, prior to the actual masking, to find an appropriate substitution value for each variable. We test each x_i variable by removing it. If the fitness immediately drops to zero—or, more practically, below a certain threshold $\delta = 10^{-6}$ —we assume the variable to behave multiplicatively; thus we set its substitution value to 1; otherwise, we set it to 0, assuming additivity.

This method is not optimal. Both benchmark functions or real-world problems may show non-linearity instead of purely additive or multiplicative behavior. For such cases, the preparation phase often becomes redundant, except for identifying variables whose substitution with 0 would completely nullify their contribution. Nonetheless, we adopt this substitution because, although not exact, it is simple, avoids additional computational complexity, and—provided the induced bias is small—approximates the effect of a variable having negligible impact on the function value.

Exempting a subset of variables from contributing to the final fitness value—i.e., actively using only a portion of them for individual evaluation—introduces an evaluation bias. This bias can be compensated by multiplying each fitness value by the inverse of the probability of retaining a variable, i.e., $1/p$ (as described below).

Following the same logic, genetic variation operators, namely crossover and mutation, are applied exclusively to the non-dropped variables. The algorithm extracts and stores their indices, ensuring that variation is introduced only on variables that effectively contribute to the fitness.

The offspring are re-evaluated on the complete decision vector (i.e., with $r = 0$) to determine their complete fitness before advancing to the next generation. This step ensures that the contribution of previously dropped variables is properly accounted for. This two-evaluation process introduces controlled randomness into the search, promoting exploration across all variables.

Notice that while we select individuals evaluated under noisy conditions after dropout, replacement is carried out using both non-noisy parents and offspring. The parents are copied after the first non-noisy evaluation. The offspring come from a second round of non-noisy evaluation. This ensures that selection-based replacement remains consistent across data. Choosing not to adopt either of these strategies could further push the algorithm toward exploration. However, this would risk favoring the parents whenever dropout performs poorly. We leave this issue for future research.

Inverse Probability Weighting for Fitness Bias Correction. Subsampling the decision variables prior to fitness evaluation introduces a systematic bias, as only a random subset of contributions is observed at each evaluation. To correct for this, we employ the Horvitz–Thompson (HT) estimator [11], a classical application of inverse probability weighting (IPW) [18] for estimating totals under probabilistic sampling. Conceptually, HT allows recovering an unbiased estimate of a target population from partial observations. By analogy, it can be applied when full data are not available and only a subset is observed [18].

In our dropout setting, each fitness evaluation is performed on a random subset of decision variables. We model this process through a binary mask $I = (I_1, \dots, I_m)$, where each indicator is sampled independently as

$$I_k \sim \text{Bernoulli}(p), \quad (2)$$

with $p = 1 - r$ the probability of retaining a variable. The resulting fitness under dropout is

$$F_{\text{drop}}(I) = \sum_{k=1}^m I_k \varphi_k(x_k), \quad (3)$$

whereas the full fitness is

$$F_{\text{full}} = \sum_{k=1}^m \varphi_k(x_k). \quad (4)$$

We assume each contribution to the fitness to be additive. This assumption is essential, as it allows each retention indicator I_k to be treated analogously to a treatment indicator $1\{A = 1\}$ in IPW. Under additivity, each retained variable contributes independently to the fitness, enabling a per-variable Horvitz–Thompson correction:

$$\hat{F} = \sum_{k=1}^m \frac{I_k}{p} \varphi_k(x_k) = \frac{1}{p} F_{\text{drop}}, \quad (5)$$

which is an unbiased estimator of the full fitness, i.e., $\mathbb{E}[\hat{F}] = F_{\text{true}}$.

Since the dropout mechanism is independent of the decision variables, the inclusion probability is constant and does not depend on the covariates. In causal inference terminology, this corresponds to a constant propensity score,

$$P(I_k = 1 \mid X) = p, \quad (6)$$

for all k . Under this assumption, the Horvitz–Thompson correction reduces to a uniform rescaling by $1/p$.

The additivity assumption in (4) naturally represents a limitation, since the contributions of decision variables may interact in a dependent and often non-linear manner. As a consequence, the proposed correction cannot fully eliminate the systematic bias introduced by subsampling. Nevertheless, the correction is extremely simple and computationally efficient, and, similarly to the assumption adopted when replacing dropped variables with fixed values (0 or 1, in Subsection 2.1), we consider the residual bias to be acceptable in our context, as the primary objective is to promote exploration rather than exact fitness recovery, especially during the early stages of the optimization process.

2.2 Population Dropout

PDrop works by stochastically inhibiting a portion of the population at each generation, according to r_p . Dropped individuals are temporarily excluded from evolution and reintegrated at the end of a generation.

After the population is fully evaluated, the evolutionary process starts by randomly excluding a percentage of individuals from the current generation. The algorithm records the indices of both the individuals kept and those discarded. Evolutionary operators are then applied only to the individuals that are kept. Afterward, the full population is rebuilt by bringing back the retained individuals, which takes the place of the usual replacement step found in standard GAs. This dropout and restoration process acts as a replacement mechanism. Unlike IDrop, this method does not use a correction factor because there is no bias in the fitness values that requires fixing.

2.3 Combined Individual & Population Dropout

IDrop and PDrop can be combined in three variants:

- At each generation, a random subset of individuals is excluded from the evolutionary process, according to r_p , and reintegrated at its end, so that IDrop operates, according to r_i , only on the non-dropped individuals (I&PDrop1).
- At each generation, a random subset of individuals is exempted from IDrop according to r_{pe} , while continuing to participate in the evolutionary process. The remaining individuals undergo per-variable dropout according to r_i (I&PDrop2). While in the previous variant all individuals participating in an evolutionary round are subject to IDrop, here some of such individuals are exempt from IDrop.
- At each generation, a random subset of individuals is excluded from the evolutionary process according to r_p , while another subset—with no overlap—is exempted from IDrop according to r_{pe} and continues to participate in the evolutionary process. The remaining individuals undergo per-variable dropout according to r_i (I&PDrop3). This is a combination of the two previous variants.

3 Experiments

Performance is evaluated in terms of the median and interquartile range (IQR) of the best-found solutions across 30 independent runs, in 12 well-known single-objective (SO) and 5 multi-objective (MO) benchmark functions. In each run, the optimization process is carried out for 1000 generations.

While SO problems performance evaluation is based on best solution's fitness, in MO two metrics are considered: Hypervolume (HV) [28], which measures the size of the portion of objective space dominated by the observed Pareto set, bounded by a reference point; Inverted Generational Distance (IGD) [24], which is a measure of convergence that averages the distance from each point on the true Pareto front to the nearest obtained solution.

For each SO benchmark function, a total of 51 experimental configurations is used, corresponding to all relevant combinations of dropout rates drawn from the set $\{0.25, 0.50, 0.75\}$. For MO functions, only 42 configurations are used, as I&PDrop2 requires excessive amount of time to complete all the necessary tests.

All functions are noise-free and unconstrained, differing in modality, scalability, separability, continuity, and differentiability. A common parameter configuration is used for each problem class, with each run having a distinct random seed (42–71) to ensure reproducibility and comparability (Table 1).

A standard GA is used as baseline (BL), and its performance is validated by comparison with reference results reported in the literature [3, 4, 27].

Statistical significance is assessed using the Wilcoxon signed-rank test. For each benchmark function, the resulting p-values are adjusted using the Benjamini–Hochberg procedure to control the false discovery rate in the 51 experimental configurations, with a significance level set to 0.05.

For each test suite, the number of methods whose performance significantly outperforms the baseline is reported. In addition, the best-performing method is highlighted together with its percentage improvement, defined as:

$$\Delta\% \text{ over baseline} = \frac{|\text{median}_{\text{baseline}} - \text{median}_{\text{method}}|}{\text{median}_{\text{baseline}}} \times 100. \quad (7)$$

A metric considered only for single-objective problems is the absolute frequency of runs, for each method and function, that converge to the target within a tolerance of 0.01 under a modified form of relative error (RE), the Hyb error [26]. RE measures the accuracy of an observed value with respect to a target, normalized by the magnitude of the target itself; however, it becomes undefined when such a magnitude is zero, which can be an issue for our purposes, as many benchmark functions have a target equal to zero [6]. The Hyb error solves such a limitation, being defined as:

$$\text{Hyb error} = \frac{|f(x_i) - f(x^*)|}{|f(x^*)| + 1}, \quad (8)$$

where $f(x_i)$ is the observed fitness and $f(x^*)$ is the target. A run is counted as converged if the Hyb error does not exceed 0.01.

In addition, to quantify exploration throughout the evolutionary process, we employ a dimension-wise diversity proposed in [2], defined as follows:

$$\text{Div}_j = \frac{1}{n} \sum_{i=1}^n |\text{median}(x^j) - x_i^j| \quad (9)$$

$$\text{Div} = \frac{1}{m} \sum_{j=1}^m \text{Div}_j, \quad (10)$$

where $\text{median}(x^j)$ is the median value of the j-th dimension across the entire population, x_i^j is the j-th dimension of the i-th individual, n is the population size and m is the number of decision variables.

This metric is used to quantify the exploration–exploitation balance through the percentages of exploration (XPL%) and exploitation (XPT%) [15], computed at each iteration as:

$$\text{XPL}\% = \frac{\text{Div}}{\text{Div}_{\max}} \times 100 \quad (11)$$

$$\text{XPT}\% = \frac{|\text{Div} - \text{Div}_{\max}|}{\text{Div}_{\max}} \times 100, \quad (12)$$

where $\text{Div}_{\max}$ is the maximum diversity value found in the whole evolution.

Table 1: Single- and multi-objective test function parameters

Parameter	SO value	MO value
# runs		30
# generations		1000
Dimensionality (n)		10, 40
Random seed	42 (+1 at each subsequent run)	
Population size		200
Crossover operator	One-Point Crossover	SBX-Bounded ^[a] ($\eta_c = 15$) ^[b]
Crossover rate		0.75
Mutation operator	Gaussian Mutation ($\mu = 0.0, \sigma = 0.5$) ^[c]	Polynomial Bounded Mutation ($\eta_m = 15$) ^[b]
Mutation rate per individual		0.10
Mutation rate per gene		0.75
Selection operator	Tournament Selection (size = 2)	NSGA2 ^[d]
Replacement operator		$\mu + \lambda$ ^[e]
Number of divisions (P) ^[f]	–	199

^[a] Simulated Binary Crossover.

^[b] η_c and η_m are the distribution indexes, which control how far offspring move from their parents after crossover and mutation, respectively. Higher values indicate higher similarity after variation.

^[c] μ and σ refer to the mean and standard deviation of Gaussian mutation.

^[d] Non-dominated Sorting Genetic Algorithm 2.

^[e] A replacement strategy consisting in selecting the best from parents (μ) and offspring (λ).

^[f] For uniform reference points.

4 Discussion

The results of the various methods on the different functions show patterns that are difficult to recognize and are particularly irregular. We analyze this by identifying the characteristics common to the functions in which a particular method performs best in terms of percentage improvement of the median compared to the baseline, in which a large number of methods achieve a median performance above the baseline, and in which a significant number of runs converge to the target (Figs. S1-S3). In addition, we compare results of different methods and their respective rate combinations. A summarized version is presented in Tables 2 and 3. Detailed data is present in Figs. S4-S27 and accompanying Tables S1-S24 for SO functions and Tables S25-S34 for MO functions.

Median percentage improvement over baseline. Median improvements over 70% occur for Ackley, Rastrigin, and Schwefel at 10 variables, and for Ackley, Griewank, and Rastrigin at 40 variables. Among the unimodal functions, on the other hand, such a significant improvement is only seen at 40 variables, especially in Ellipsoid separable and Rosenbrock. The rest of the functions achieve a modest or negligible improvement at both dimensionalities.

Ackley and Rastrigin are therefore the only ones whose improvement remains significant even with increased dimensionality. The two functions have much in common, especially the fact that they possess a regularly structured and potentially exploitable multimodality. In both cases, the best methods are the same: at 10 variables, IDrop ($r_i = 0.75$); at 40 variables, I&PDrop3 ($r_i = 0.25, r_p = 0.25, r_{pe} = 0.50$; Fig. 2). Schwefel is a typically deceptive

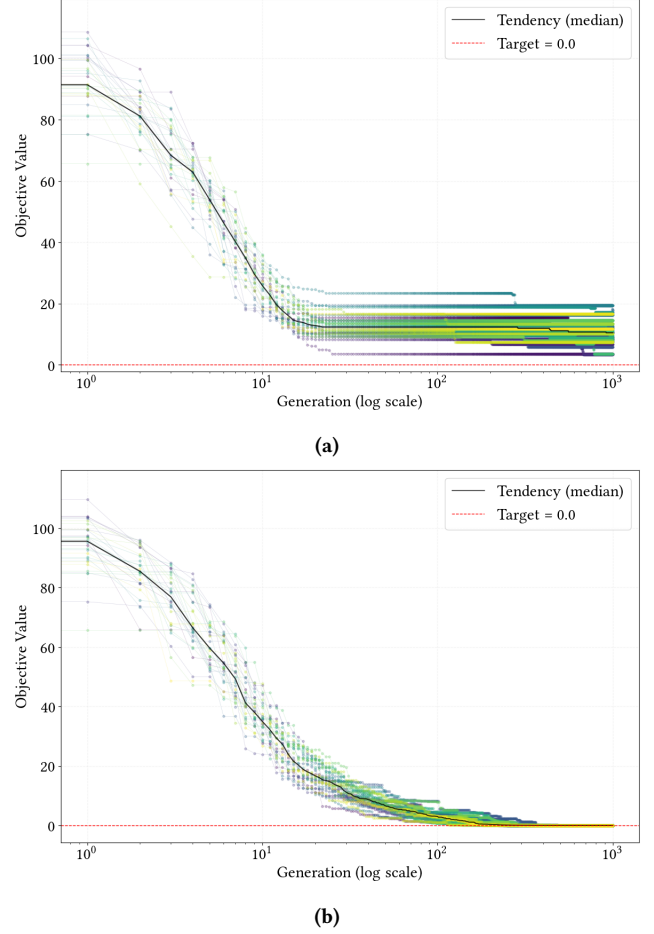


Figure 2: Optimization trajectories on Rastrigin (30 runs, 10 variables each): (a) baseline; (b) IDrop ($r_i = 0.75$).

function, designed to mislead optimization by containing multiple promising regions; in fact, it contains approximately 2^n major minima near the edges of the search space, with the global optimum close to the boundary. With 10 variables, the best method, I&PDrop2 ($r_i = 0.50, r_{pe} = 0.50$), converges particularly well compared to the baseline, with a median improvement over the baseline of 6 orders of magnitude, although the interquartile range is very high; with 40 variables, the improvement is much more modest (IDrop, $r_i = 0.25$).

Griewank, like Ackley and Rastrigin, has a regularly distributed multimodality, but unlike them, it is smoother, with a large number of moderately deep local minima. With it, there is an increase in the median percentage improvement from 10 variables (IDrop, $r_i = 0.25$) to 40 (I&PDrop2, $r_i = 0.25, r_{pe} = 0.25$), indicating that the increase in complexity given by higher dimensionality, in this case, is attenuated.

Ellipsoid separable and Rosenbrock are typically ill-conditioned functions, i.e. with narrow curves/different slopes for each direction, making it difficult for many algorithms to progress smoothly towards the optimum, especially those based on gradients or those that do not adapt well to the shape of the valley. In both 10-variable

Table 2: Best methods on single-objective multimodal and unimodal test functions

f	$f(x^*)$	n	BL Median (IQR)	Best Method	r_i	r_p	r_{pe}	Median (IQR)	p (BH)	$\Delta\%$	# better/51
Ackley [1]	0.00e+00	10	3.78e-01 (1.17e-01)	IDrop	0.75	–	–	6.26e-03 (3.73e-03)	3.80e-09	98.34%	49
		40	1.52e+01 (1.07e+00)	I&PDrop3	0.25	0.25	0.50	6.92e-01 (2.04e-01)	3.80e-09	95.45%	46
Gallagher 101 peaks [8] (bbob-f21-01)	4.08e+01	10	4.28e+01 (1.43e+00)	–	–	–	–	–	–	–	0
		40	4.28e+01 (4.47e+00)	IDrop	0.50	–	–	4.24e+01 (3.45e+00)	3.64e-02	0.01%	2
Griewank [9]	0.00e+00	10	3.33e+00 (2.04e+00)	IDrop	0.25	–	–	1.96e+00 (1.07e+00)	1.92e-03	41.14%	4
		40	4.34e+01 (2.54e+01)	I&PDrop2	0.25	–	0.25	8.96e+00 (8.13e+00)	9.50e-09	79.35%	9
Katsuura [12] (bbob-f23-01)	6.87e+00	10	8.33e+00 (2.95e-01)	IDrop	0.75	–	–	7.61e+00 (5.17e-01)	2.48e-06	8.64%	41
		40	1.10e+01 (5.08e-01)	PDrop	–	0.25	–	1.03e+01 (3.40e-01)	5.59e-09	6.36%	44
Lunacek bi-Rastrigin [13] (bbob-f24-01)	1.03e+02	10	1.48e+02 (1.08e+01)	I&PDrop1	0.75	0.25	–	1.31e+02 (6.08e+00)	1.40e-04	11.49%	42
		40	5.69e+02 (6.05)+01	PDrop	–	0.25	–	3.93e+02 (1.20e+01)	4.07e-08	30.93%	48
Rastrigin [17]	0.00e+00	10	1.05e+01 (5.49e+00)	IDrop	0.75	–	–	5.25e-03 (5.84e-03)	5.00e-09	99.95%	50
		40	1.94e+02 (4.07e+01)	I&PDrop3	0.25	0.25	0.50	5.82e+01 (1.44e+01)	2.50e-09	70.00%	50
Schaffer F7, Cond. 10 [5] (bbob-f17-01)	-1.69e+01	10	-1.65e+01 (2.63e-01)	I&PDrop3	0.75	0.25	0.50	-1.68e+01 (5.88e-02)	1.68e-06	1.82%	20
		40	-1.11e+01 (1.24e+00)	I&PDrop3	0.50	0.25	0.50	-1.60e+01 (1.54e-01)	3.96e-09	44.14%	47
Schwefel [21]	0.00e+00	10	1.18e+02 (1.18e+02)	I&PDrop2	0.50	–	0.50	3.94e-04 (1.18e+02)	2.14e-05	100.00%	5
		40	2.93e+03 (9.63e+02)	IDrop	0.25	–	–	1.37e+03 (7.06e+02)	1.90e-08	53.24%	10
Weierstrass [5] (bbob-f16-01)	7.13e+01	10	7.43e+01 (1.49e+00)	I&PDrop3	0.25	0.25	0.75	7.29e+01 (1.48e+00)	4.86e-03	1.88%	8
		40	9.16e+01 (3.72e+00)	I&PDrop3	0.75	0.25	0.75	8.23e+01 (3.41e+00)	3.05e-04	10.15%	12
Ellipsoid [5] (bbob-f10-01)	-5.49e+01	10	2.60e+03 (3.04e+03)	–	–	–	–	–	–	–	0
		40	2.15e+05 (2.07e+05)	IDrop	0.75	–	–	1.07e+05 (3.60e+04)	4.13e-09	50.23%	14
Ellipsoid separable [5] (bbob-f02-01)	-2.10e+02	10	-1.56e+02 (4.15e+01)	IDrop	0.75	–	–	-2.09e+02 (1.99e+00)	7.31e-09	33.97%	21
		40	5.84e+04 (3.62e+04)	I&PDrop2	0.75	–	0.25	1.85e+02 (2.43e+02)	2.50e-09	99.68%	28
Rosenbrock [19]	0.00e+00	10	1.05e+01 (2.14e+00)	IDrop	0.75	–	–	7.47e+00 (6.18e-01)	2.37e-08	28.86%	49
		40	3.94e+02 (1.20e+02)	I&PDrop3	0.75	0.25	0.25	6.07e+01 (6.16e+00)	2.37e-09	84.59%	50

Table 3: Best HV and IGD results on multi-objective test functions

f	n	Metric	BL Median (IQR)	Best Method	r_i	r_p	r_{pe}	Median (IQR)	p (BH)	$\Delta\%$	# better/42
ZDT1 [27]	10	HV	3.37e+00 (1.10e-02)	I&PDrop3	0.25	0.75	0.25	9.06e+00 (1.16e+00)	2.44e-09	168.84%	39
		IGD	2.35e-05 (2.62e-06)	–	–	–	–	–	–	0%	0
	40	HV	3.48e+00 (6.06e-02)	I&PDrop1	0.25	0.50	–	1.00e+01 (5.28e-01)	2.00e-09	187.36%	39
		IGD	6.64e-03 (3.99e-03)	IDrop	0.75	–	–	4.95e-04 (9.81e-05)	1.91e-09	92.54%	3
ZDT2 [27]	10	HV	3.71e+00 (2.24e-03)	I&PDrop1	0.50	0.75	–	8.73e+00 (1.39e+00)	2.00e-09	135.31%	37
		IGD	2.56e-05 (2.68e-06)	–	–	–	–	–	–	0%	0
	40	HV	3.82e+00 (5.20e-02)	I&PDrop1	0.50	0.75	–	1.06e+01 (1.65e+00)	2.00e-09	177.49%	38
		IGD	3.23e-02 (1.60e-02)	IDrop	0.75	–	–	7.04e-04 (2.20e-04)	1.91e-09	97.82%	3
ZDT3 [27]	10	HV	2.96e+00 (1.29e-02)	I&PDrop3	0.25	0.75	0.25	8.86e+00 (1.25e+00)	2.00e-09	199.32%	39
		IGD	3.57e-05 (3.75e-06)	–	–	–	–	–	–	0%	0
	40	HV	3.06e+00 (7.96e-02)	I&PDrop1	0.50	0.75	–	9.54e+00 (6.90e-01)	1.96e-09	211.76%	39
		IGD	2.87e-03 (2.45e-03)	IDrop	0.75	–	–	4.08e-04 (7.03e-05)	1.91e-09	85.78%	3
ZDT4 [27]	10	HV	5.35e+00 (1.81e+00)	I&PDrop1	0.25	0.75	–	2.05e+02 (5.13e+01)	2.00e-09	3.73e+03%	39
		IGD	2.77e+00 (1.84e+00)	IDrop	0.75	–	–	1.23e-03 (1.75e-03)	1.91e-09	99.96%	3
	40	HV	7.87e+01 (2.05e+01)	I&PDrop3	0.25	0.50	0.25	1.04e+03 (1.42e+02)	2.00e-09	1.22e+03%	39
		IGD	7.66e+01 (2.03e+01)	IDrop	0.25	–	–	3.19e+01 (9.60e+00)	1.95e-09	59.47%	3
ZDT6 [27]	10	HV	3.72e+00 (6.24e-02)	I&PDrop1	0.50	0.50	–	1.76e+01 (6.87e-01)	1.86e-09	373.12%	39
		IGD	1.28e-03 (4.12e-04)	IDrop	0.25	–	–	9.54e-04 (2.17e-04)	4.95e-07	25.47%	3
	40	HV	9.12e+00 (6.29e-01)	I&PDrop1	0.25	0.25	–	1.86e+01 (5.77e-01)	1.91e-09	103.95%	39
		IGD	2.78e+00 (2.95e-01)	IDrop	0.25	–	–	2.52e-01 (2.16e-02)	1.91e-09	90.94%	3

cases, the best method (IDrop, $r_i = 0.75$) improves only slightly

compared to the baseline. With 40 variables, there is a significant

percentage improvement with I&PDrop2 ($r_i = 0.75$, $r_{pe} = 0.25$) and I&PDrop3 ($r_i = 0.75$, $r_p = 0.25$, $r_{pe} = 0.25$), respectively.

The rest of the functions obtain null or negligible results. Specifically: in both dimensions, Gallagher 101 peaks, Weierstrass, Katsuura; especially at 10 variables, Lunacek bi-Rastrigin, Schaffer F7 Condition 10 and Ellipsoid. In a multimodal context, Weierstrass and Katsuura exhibit irregular ruggedness across scales and repetition, while Gallagher 101 peaks has ruggedness coming from many isolated peaks, with no coherent funnel or global attractor. All three lack a strong exploitable global structure. Schaffer F7 Condition 10 does not exhibit a statistically significant improvement; however, with 10 variables, the median performance is very close to the target value, within a tolerance of approximately 1.

Separability does not appear to be a determining factor in the success or failure of dropout-based methods. Although an assumption of additivity is implicitly made (see Subsection 2.1), the methods perform well not only on the tested separable functions (Rastrigin and separable Ellipsoid), but also on several non-separable functions (Tables 2 and 3).

Number of combinations outperforming the baseline (median). All functions show consistent agreement at 10 and 40 variables, except for Schaffer F7 Condition 10 and Ellipsoid. This number is always slightly higher at 40 variables, except for Ackley, suggesting that, regardless of the convergence results, the increase in dimensionality causes the median performance to decrease faster on the baseline than on the applied dropout methods. Finally, we observe that this number is consistently high, often close to 51. This occurs both for functions and dimensionalities where the percentage improvement is substantial—such as Ackley, Rastrigin, and Rosenbrock—and for cases in which the improvement achieved by the best method is negligible, as observed for Katsuura, Lunacek bi-Rastrigin, and Schaffer F7 Condition 10 at 40 variables. This suggests that, for these functions, there is no marked preference for one method over all others, but that almost all of them manage to improve, albeit slightly, the median fitness of the baseline. In contrast, in Weierstrass and Gallagher 101 peaks, we observe poor results under this metric, as before, whereas Griewank and Schwefel degrade despite good performance on $\Delta\%$ (Tables 2 and 3).

Absolute frequency of convergent runs per function. If we consider instead the number of convergent runs, out of 30 per function, we find results that appear to contrast with the previous ones (Figs S1–S3). In fact, only some functions appear to have a number of convergent runs greater than zero. In particular, at 10 variables, we note that Schwefel, Schaffer F7 Condition 10 and Gallagher 101 peaks show, out of a total of $51 \times 30 = 1530$ runs, approximately 18%, 16% and 6% of convergent runs within the pre-established Hyb error threshold, respectively. Others that obtain a number of convergent runs greater than zero are Weierstrass (3%), Rastrigin (2%) and Ackley (2%). Surprisingly, Weierstrass, which showed poor results in terms of median percentage improvement and number of rate and strategy combinations outperforming the baseline on a median basis, reports at least 1 convergent run across 18/51 method combinations, while its baseline brings no convergent runs.

The differences between modality and dimensionality of problem classes become much more pronounced here. In fact, we note that in unimodal problems with 10 variables, only Ellipsoid separable

achieves a small percentage of convergent runs, equal to 7%, with IDrop ($r_i = 0.75$) as the best method. If, on the other hand, the dimensionality is increased, convergence is never achieved in many of the functions. Only in Gallagher-101 peaks, with 40 variables, is a convergence rate of 5% achieved.

In particular, for Gallagher 101 peaks with 10 variables, the baseline achieves a number of convergent runs comparable to that of several dropout-based method combinations. In contrast, at 40 variables the baseline fails to produce any convergent runs, whereas multiple dropout configurations achieve convergence in approximately 1/3 of the runs—most notably IDrop ($r_i = 0.50$ or $r_i = 0.75$) and I&PDrop2 ($r_i = 0.75$, $r_{pe} = 0.25$ or $r_i = 0.75$, $r_{pe} = 0.50$).

31/51 dropout method combinations find at least 1 convergent run. Some combinations manage to find a fair number of convergent runs for several of the functions mentioned, such as IDrop ($r_i = 0.75$), with as many as 72 convergent runs, or I&PDrop2 ($r_i = 0.50$, $r_{pe} = 0.50$), with 39.

We also note that, unlike Ackley and Rastrigin, where IDrop ($r_i = 0.75$) is significantly more impactful than the other combinations, causing almost all runs to converge, the other functions show a small but still highly represented number in almost all 31 combinations with at least 1 convergent run, with variable peaks. For example, Gallagher 101 peaks and Schwefel seem to prefer I&PDrop2; on the contrary, Schaffer F7 Condition 10 prefers I&PDrop3; Weierstrass, on the other hand, does not show any significant peaks.

Emergent Patterns Across Metrics. Results show that dropout effects on exploration highly depend on the strategy: per-individual deactivation methods (i.e., PDrop, I&PDrop1 and I&PDrop3) shift exploration dominance from 10 to 100 generations, whereas the absence of such a mechanism yields to balances closer to the baseline. In general we observe that more complex functions are those in which the XPL-XPT balance most benefits from using dropout, an effect that is particular clear at higher dimensions (Figs. S28–S51). Unfortunately, this improvement in the XPL-XPT balance does not always correlate with a better performance, and this should further investigated.

In most functions and dimensionalities, a modest number of dropout combinations have a decidedly positive effect in terms of both performance and robustness compared to a simple GA. This is especially true for multimodal functions with a highly exploitable global structure, such as Ackley and Rastrigin at both dimensionalities. For these, the best overall result is obtained with 10 variables with IDrop ($r_i = 0.75$), as it also converges in nearly all runs. Breaking the interdependence of variables, even in non-separable contexts, proves to contribute beneficially when the risk of losing global directional information is limited, thus helping overcoming local optima efficiently.

On the other hand, functions with little or no exploitable global structure, such as Gallagher 101 peaks, Weierstrass, Ellipsoid and Katsuura (except for the number of best combinations) show little or no improvement when compared to the baseline at both dimensionalities.

Moreover, increasing dimensionality affects dropout-based methods much less than the baseline in all functions. Nevertheless, no

sufficiently acceptable convergence rates are obtained. This is particularly evident for Griewank (in terms of $\Delta\%$ only), Lunacek bi-Rastrigin, Schaffer F7 Condition 10, Weierstrass, Ellipsoid separable, and Rosenbrock.

In such high dimensional contexts—where search agents are more likely to get stuck in local optima—I&PDrop2 and I&PDrop3 are preferred. Here, they lead to positive or decidedly positive median results, even if they never manage to achieve convergence (except in Gallagher 101 peaks).

Moving to multi-objective problems, the distinction between the effects of the various methods becomes much more evident.

The presence of PDrop, especially when combined with IDrop, leads to significant improvements in terms of HV, indicating enhanced dominance and diversity of the approximated Pareto front with respect to the theoretical one, but worse convergence in both dimensions. Conversely, the sole application of IDrop leads to excellent results in terms of IGD, and thus convergence, while achieving slightly worse or comparable HV values.

Looking at the distributions of the best 20 solutions in each function, we note that, at 40 variables only, while the baseline is able to bring only the first objective close to the target and fails to do so with the second, IDrop—particularly at high rates (0.75)—achieves a successful convergence with both. This is true for all the tested instances of the ZDT suite, indicating its robustness to different characteristics of convexity, concavity, discontinuity, multimodality, and irregularity of the Pareto Front.

5 Conclusion

In this work, we investigated the impact of three diversity inducing dropout-based mechanisms—and their respective variants—on the performance of GAs using 12 well-known single-objective and 5 multi-objective benchmark functions. The experimental analysis reveals a blurred and highly context-dependent picture.

Across single-objective problems, dropout-based methods demonstrate clear advantages over a standard GA in terms of median performance in landscapes characterized by a regular and exploitable global structure, such as Ackley and Rastrigin. In these cases, individual level dropout, especially at high rates, effectively mitigates premature convergence and, as long as the dimensionality does not grow excessively, facilitates consistent convergence across runs. More complex or deceptive landscapes benefit from combined strategies that integrate dropout combinations, especially at high-dimensionalities, although convergence still remains limited.

Conversely, functions lacking a coherent global structure or exhibiting highly irregular ruggedness show little or no benefit from dropout mechanisms when evaluated solely on median performance. Nevertheless, an analysis based on the absolute number of convergent runs highlights that in some cases several dropout configurations are able to achieve at least one converging run in problems where the baseline fails entirely.

In multi-objective optimization, the distinction between dropout strategies becomes more pronounced. Individual-level dropout favors convergence-oriented metrics such as IGD, while dropout combinations enhance dominance and diversity, as reflected by improved HV values. These results suggest that different dropout

mechanisms bias the search process in fundamentally different ways, making them suitable for distinct optimization goals.

Overall, the findings confirm that dropout can act as an effective mechanism for modulating the exploration–exploitation balance in EAs, improving robustness to increased dimensionality, and, in selected problem classes, substantially enhancing performance.

The most evident limitation is operational: the complete re-evaluation of the entire individuals at the end of each cycle required by the per-variable dropout, as implemented here, significantly slows down the execution and can become a serious issue, especially for high-dimensional or multi-objective problems. On the other hand, in PDrop a considerable number of evaluations are avoided; as a result, I&PDrop1 and I&PDrop3 exhibit a much lower computational cost.

Moreover, IDrop relies on stochastic 0/1 substitution, limiting its applicability to combinatorial or integer optimization. In addition, neutral-value substitution introduces bias that is difficult to disentangle from other effects, particularly for black-box problems.

Another important limitation emerges from the results: not only do none of the dropout methods perform well in all contexts—which is actually to be expected in light of the No Free Lunch Theorem [25]—but it is also particularly difficult to identify which problem characteristics make a given dropout strategy, at a specific rate, more suitable than others. Thus, the blind application of any dropout-based method is inadequate without explicit control over the optimization process response to such strategies.

An *a priori* assessment may guide the choice of dropout mechanism, while for black-box functions an online or *a posteriori* assessment may be preferable. Such evaluations should characterize the effects of dropout on the exploration–exploitation trade-off during evolution, as well as on varying operators such as crossover and mutation. We leave a study of this direction to future work.

Further testing is needed to validate the evidence gathered here, especially in functions with a highly exploitable global landscape structure, varying dimensionality, separability and deception.

Other ways of encouraging exploration through dropout could be considered. For example, to apply it at the objective level in problems with a number of objectives ≥ 3 , or to test dynamic or adaptive rates based on some computations.

Exploring other parameter settings instead of those used here could also be relevant. This would help determine how much the methods rely on the starting parameter configurations.

Finally, future work may extend dropout to state-of-the-art continuous optimizers, where it could serve as a perturbation operator of internal mechanisms to improve diversity and exploration in the space of solutions.

Supplementary Materials, Code and Data Availability

Supplementary Materials are available at <https://doi.org/10.5281/zenodo.19497905>. Code and data are available at <https://github.com/deapsleep2026/deapsleep> and <https://doi.org/10.5281/zenodo.18383258>, respectively.

References

- [1] David H. Ackley. 1987. *A connectionist machine for genetic hillclimbing*. Kluwer Academic Publishers, USA.
- [2] Shi Cheng, Yuhui Shi, Quande Qin, Qingyu Zhang, and Ruibin Bai. 2014. Population Diversity Maintenance In Brain Storm Optimization Algorithm. *Journal of Artificial Intelligence and Soft Computing Research (JAISCR)* 4 (10 2014), 83–97. doi:10.1515/jaiscr-2015-0001
- [3] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. doi:10.1109/4235.996017
- [4] Iason Digalakis, Konstantinos Margaritis, and Konstantinos G. Margaritis. 2001. On Benchmarking Functions for Genetic Algorithms. *International Journal of Computer Mathematics* 77 (2001). doi:10.1080/00207160108805080
- [5] Steffen Finck, Nikolaus Hansen, Raymond Ros, and Anne Auger. 2009. *Real-Parameter Black-Box Optimization Benchmarking 2009: Noiseless Functions Definitions*. Technical Report RR-6829. INRIA. <https://inria.hal.science/inria-00362633v2/document> Updated version as of February 2019.
- [6] William Ford. 2014. *Numerical Linear Algebra with Applications using MATLAB*.
- [7] Tobias Friedrich, Pietro S Oliveto, Dirk Sudholt, and Carsten Witt. 2009. Analysis of diversity-preserving mechanisms for global exploration. *Evolutionary Computation* 17, 4 (2009), 455–476. doi:10.1162/evco.2009.17.4.17401
- [8] M. Gallagher and Bo Yuan. 2006. A general-purpose tunable landscape generator. *IEEE Transactions on Evolutionary Computation* 10, 5 (2006), 590–603. doi:10.1109/TEVC.2005.863628
- [9] A. O. Griewank. 1981. Generalized Descent for Global Optimization. *Journal of Optimization Theory and Applications* 34, 1 (1981), 11–39. doi:10.1007/BF00933356
- [10] Geoffrey E. Hinton, Nitish Srivastava, Alex Krizhevsky, Ilya Sutskever, and Ruslan R. Salakhutdinov. 2012. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580* (2012). <https://arxiv.org/abs/1207.0580> Published July 3 2012.
- [11] D. G. Horvitz and D. J. Thompson. 1952. A Generalization of Sampling Without Replacement From a Finite Universe. *J. Amer. Statist. Assoc.* 47, 260 (1952), 663–685. <http://www.jstor.org/stable/2280784>
- [12] Hidefumi Katsuura. 1991. Continuous Nowhere-Differentiable Functions — an Application of Contraction Mappings. *The American Mathematical Monthly* 98, 5 (1991), 411–416. arXiv:<https://doi.org/10.1080/00029890.1991.12000778> doi:10.1080/00029890.1991.12000778
- [13] Monte Lunacek, Darrell Whitley, and Andrew Sutton. 2008. The Impact of Global Structure on Search. 498–507. doi:10.1007/978-3-540-87700-4_50
- [14] Z. Michalewicz. 1996. *Genetic Algorithms + Data Structures = Evolution Programs*. Springer-Verlag, New York, NY.
- [15] Bernardo Morales-Castañeda, Daniel Zaldivar, Erik Cuevas, Fernando Fausto, and Alma Rodriguez. 2020. A better balance in metaheuristic algorithms: Does it exist? *Swarm and Evolutionary Computation* 54 (2020), 100671. doi:10.1016/j.swevo.2020.100671
- [16] Hari Mohan Pandey, Ankit Chaudhary, and Deepti Mehrotra. 2014. A comparative review of approaches to prevent premature convergence in GA. *Applied Soft Computing* 24 (2014), 1047–1077. doi:10.1016/j.asoc.2014.08.025
- [17] L. A. Rastrigin. 1974. *Systems of Extremal Control*. Mir, Moscow. Original in Russian: “Sistemy ekstremal’nogo upravleniya”.
- [18] James M. Robins, Andrea Rotnitzky, and Lue Ping Zhao. 1994. Estimation of Regression Coefficients When Some Regressors are not Always Observed. *J. Amer. Statist. Assoc.* 89, 427 (1994), 846–866. arXiv:<https://doi.org/10.1080/01621459.1994.10476818> doi:10.1080/01621459.1994.10476818
- [19] H. H. Rosenbrock. 1960. An Automatic Method for Finding the Greatest or Least Value of a Function. *Comput. J.* 3, 3 (01 1960), 175–184. arXiv:<https://academic.oup.com/comjnl/article-pdf/3/3/175/988633/030175.pdf> doi:10.1093/comjnl/3.3.175
- [20] Bruno Sareni and Laurent Krähenbühl. 1998. Fitness sharing and niching methods revisited. *IEEE Transactions on Evolutionary Computation* 2, 3 (1998), 97–106. doi:10.1109/4235.735432 Accessed: 2025-12-11.
- [21] Hans-Paul Schwefel. 1981. *Numerical Optimization of Computer Models*. John Wiley & Sons, Chichester, England. English translation of German original “Numerische Optimierung von Computer-Modellen”, Basel 1977.
- [22] Carlos Segura, Carlos Coelho, Eduardo Segredo, and Arturo Hernandez-Aguirre. 2015. A Novel Diversity-Based Replacement Strategy for Evolutionary Algorithms. *IEEE Transactions on Cybernetics* 46 (12 2015), 1–14. doi:10.1109/TCYB.2015.2501726
- [23] Dirk Sudholt. 2020. *The Benefits of Population Diversity in Evolutionary Algorithms: A Survey of Rigorous Runtime Analyses*. Springer International Publishing, Cham, 359–404. doi:10.1007/978-3-030-29414-4_8
- [24] David A. Van Veldhuizen and Gary B. Lamont. 1998. Evolutionary Computation and Convergence to a Pareto Front. In *Late Breaking Papers at the Genetic Programming 1998 Conference*. Stanford University Bookstore, 221–228.
- [25] David H. Wolpert and William G. Macready. 1997. No Free Lunch Theorems for Optimization. *IEEE Transactions on Evolutionary Computation* 1, 1 (1997), 67–82. doi:10.1109/4235.585893
- [26] Peichen Xie. 2024. Hyb Error: A Hybrid Metric Combining Absolute and Relative Errors. arXiv:2403.07492 [math.NA] <https://arxiv.org/abs/2403.07492>
- [27] Eckart Zitzler, Kalyanmoy Deb, and Lothar Thiele. 2000. Comparison of Multiobjective Evolutionary Algorithms: Empirical Results. *Evolutionary Computation* 8, 2 (2000), 173–195. doi:10.1162/106365600568202
- [28] Eckart Zitzler and Lothar Thiele. 1998. Multiobjective optimization using evolutionary algorithms — A comparative case study. In *Parallel Problem Solving from Nature — PPSN V*, Agoston E. Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 292–301.
- [29] M. Črepinšek, S.-H. Liu, and M. Mernik. 2013. Exploration and Exploitation in Evolutionary Algorithms: A Survey. *Comput. Surveys* 45 (2013), Article No. 35. doi:10.1145/2480741.2480752